

Base System Installation Guide

The following steps describe how to build a web server on a Raspberry Pi. The steps below configure the Raspberry Pi to serve web pages from the user's document root folder. When user html folders are enabled, Apache looks for the files in the user's **public_html** folder. For example, a browser request **http://raspi.local/~pi/myDocument.html** would cause the host **raspi.local** to look for **myDocument.html** in the folder **/home/pi/public_html**. With the setup in the steps below, documents can also be served virtually, as if from the host's **/var/www** folder. For example, **http://raspi.local/myDocument.html** would also cause Apache to look for **myDocument.html** in the folder **/home/pi/public_html**.

Part A. Install the Raspbian OS on a SD Card

1. If upgrading from a previous Raspbian version, first backup the user home folder by running

```
zip -ry pi_bak.zip /home/pi/*
```

Copy the **pi_bak.zip** file to a thumb drive or some other external storage media.

2. For Raspbian Bookworm, Trixie, and beyond use the Raspberry Pi Imager tool which can be downloaded from <https://www.raspberrypi.com/software/>. Follow instructions on the web site.
3. Note that the imager will not run on Ubuntu based systems without the libxcl-cursor0 library. Install the library (if necessary) by running

```
sudo apt install libxcb-cursor0
```

4. Insert the SD card into a card reader and insert the card reader into a Linux PC.
5. Run the imager from a terminal window in the Linux PC.

```
sudo ./{imager file name}
```

6. Select the Raspberry Pi device that will be used. Click NEXT.
7. Select the desired Raspbian OS. Note that the Pi Zero should use only the Lite version of Raspbian. The Lite version does not include support for the Raspbian desktop, which is not needed for headless operation. To use the Lite version click "Raspbian Pi OS (other)". Then select "Raspbian Pi OS Lite". Click NEXT.
8. Select the storage device (SD card reader) to which the imager will copy the selected Raspbian OS. Exercise care as several devices might appear in the list. Be sure to select the correct device. Click NEXT.
9. To use the Raspberry Pi without an attached keyboard and display (headless operation) you must complete the following steps. These steps are in the Customisation section of the imager. If setting up the Raspberry Pi for use with a keyboard and display click "SKIP

Base System Installation Guide

CUSTOMISATION" and skip to step 15.

10. Enter a host name for the Raspberry Pi. E.g. "pizerodev". Click NEXT.
11. Select the capital city of your country, your timezone, and your keyboard layout. Click NEXT.
12. Enter a user name. The following steps assume you entered **pi** as the username.
13. Enter a password for the user. Confirm the password entered and click NEXT.
14. If connecting through a wifi network, select the type of network, enter the SSID and passphrase. If setting up a headless Raspberry Pi, take care to get this step right. Click NEXT.
15. For headless operation turn on "Enable SSH". Select "Use password authentication". This can be changed later on. Optional steps below will install ssh keys. Click NEXT.
16. Turn off "Enable Raspberry Pi Connect" Click NEXT.
17. Review the summary section and go back and make changes as necessary. Select WRITE. Select "I UNDERSTAND, ERASE AND WRITE".
18. When the imager finishes writing to the SD card, remove the SD card from the PC.

Part B. Install Server Components

1. Insert the SD card into the Pi and connect the Pi to its power source. The Pi will boot up.
2. Once the Pi has booted, on your computer open a terminal window and run the command

```
ssh pi@{your hostname}.local
```

For host name use the host name you entered in step 10. Enter the password set in step 13 above.

3. If you chose to configure your Raspberry Pi with a keyboard and monitor, boot up the Raspberry Pi and follow the prompts to configure your Raspberry Pi. Once you have configured your Raspberry Pi, open a terminal window and continue with step 4.
4. [Optional] To enter sudo commands without having to enter a password each time, first navigate to **/etc/sudoers.d** then create a file **010_pi-nopasswd** as follows

```
sudo su  
cd /etc/sudoers.d  
echo "pi ALL=(ALL) NOPASSWD: ALL" > 010_pi-nopasswd  
exit
```

5. Update the package database

Base System Installation Guide

sudo apt update

6. [Optional] Run all software updates

sudo apt upgrade
sudo reboot

7. [Optional] Install vim editor

sudo apt install vim

vim is an editor that makes it easy to make changes in configuration files. Alternatively you can use **nano** or some other terminal based editor of your choice.

8. [Optional] On the client computer, create an ssh key pair. In the Raspberry Pi user **pi** home folder, create a folder named **.ssh**. In the **.ssh** folder create a file named **authorized_keys**, and copy the public key to it. For more information on how to create and use key pairs see <https://docs.github.com/en/free-pro-team@latest/github/authenticating-to-github/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agent> .
9. [Optional] For added security turn off password authentication. You must have set up ssh keys as discribed in step 8. *Do not do this step unless you are familiar with ssh keys and how they work.* You will not be able to login to the Pi with a password. The above enhances security when using secure shell (ssh) to access the Raspberry Pi. Backup and then modify **/etc/ssh/sshd_config** as follows

```
# changed {date} by admin to improve security
#PasswordAuthentication yes
PasswordAuthentication no
```

10. Note that all procedures below that involve editing a system file will have to be done acting as superuser. For example, to edit the file in step 11, first backup the file to be edited and then edit the file

```
cd /boot/firmware
sudo cp config.txt config_bak.txt
sudo vi config.txt
```

It is a good practice to always back up system files before editing them.

11. [Optional] If they will not be used, disable WiFi and Bluetooth. Backup and then modify **/boot/firmware/config.txt** by adding the following two lines to the end of the file:

```
dtoverlay=pi3-disable-wifi
dtoverlay=pi3-disable-bt
```

and run the once off command:

sudo systemctl disable hciuart

Note that when using a headless Pi Zero you will need a USB mini to Ethernet adapter if not using wifi to access the Pi Zero. Otherwise do not disable wifi function of the Pi Zero.

12. Setup the temporary file system (tmpfs) by backing up and then modifying **/etc/fstab**. Add the following lines to the bottom of the file.

```
# Store log files in ram to reduce stress on the SD card due to frequent writes.
tmpfs /tmp tmpfs nodev,nosuid,size=5m 0 0
tmpfs /var/tmp tmpfs defaults,noatime,nosuid,size=4m 0 0
tmpfs /var/log tmpfs defaults,noatime,nosuid,mode=0755,size=10m 0 0
tmpfs /var/spool/mqueue tmpfs defaults,noatime,
    nosuid,mode=0700,gid=12,size=1m 0 0
```

Highly recommended, this step configures the Raspberry Pi to store all log files and temporary files in RAM. Unless configured to use an external hard drive, the Raspberry Pi mounts the root file system to the SD card. Log files and temporary files are frequently written to the file system, resulting in wear on the SD card. Storing these files in RAM saves the SD card from such wear. Note that all log files and temporary files are lost upon reboot or power down.

13. Reboot the Raspberry Pi.
14. Use ssh to login as user **pi** with the password set in step 13 in section A

ssh pi@{your host name}.local

15. Install rrdtool

sudo apt install rrdtool

rrdtool maintains round-robin databases and generates charts for display in web pages. **rrdtool** can run on the same host as **Maria** (mySql). However they are not interoperable with each other.

16. Install LAMP. This is the standard Linux web server “stack”. For more information about installing LAMP see <https://www.raspberrypi.org/documentation/remote-access/web-server/apache.md> . Run the following commands

```
Apache
=====
sudo apt install apache2 -y
sudo a2enmod rewrite
sudo systemctl restart apache2

PHP
===
sudo apt install php libapache2-mod-php -y
sudo systemctl restart apache2
```

MySQL [Optional]

=====

sudo apt install mariadb-server mariadb-client php-mysql -y

17. Backup and then modify **/etc/apache2/mods-available/userdir.conf** to allow user **.htaccess** files. For example,

```
# changed {date} by admin to allow user .htaccess file
#AllowOverride FileInfo AuthConfig Limit Indexes
AllowOverride All
```

This enables apache to use the **.htaccess** file in the user's document root folder.

18. Enable user directories in Apache by running

sudo a2enmod userdir

19. Enable php in Apache by running

sudo a2enmod php8.4

Note that your version of php may be different than 8.4.

20. Backup and then modify **/etc/apache2/mods-available/php8.4.conf** to allow PHP scripts to run in user directories by commenting the lines at bottom of the file. The bottom of the file should look like

```
# changed {date} by admin to enable user php scripts
#<IfModule mod_userdir.c>
#   <Directory /home/*/public_html>
#       php_admin_flag engine Off
#   </Directory>
#</IfModule>
```

21. Create a user **public_html** folder in the **pi** home folder.

mkdir public_html

22. Allow user group and others access to the user **pi** home folder.

chmod 711 /home/pi

23. Backup and then modify **/etc/apache2/sites-available/000-default.conf** to make the pi user **public_html** folder the web server document root. For example,

```
# changed {date} by admin to make user pi public_html folder
# the html document root
#DocumentRoot /var/www/html
```

Base System Installation Guide

DocumentRoot /home/pi/public_html

This makes the **pi** user public_html folder the document root for the Raspberry Pi. For example a client request **http://raspi.local/myDocument.html** would cause Apache to look for the document in the folder **/home/pi/public_html**.

24. Backup and then modify **/etc/apache2/envvars** to create Apache logs in **tmpfs**. Add the following lines at the top of the file

```
# added {date} by admin to allow apache to use tmpfs for log files
if [ ! -d /var/log/apache2 ]; then
    mkdir /var/log/apache2
fi
if [ ! -d /var/log/mysql ]; then
    mkdir /var/log/mysql
fi
```

Without these lines Apache will fail to start when the system reboots. When the system reboots the folders **/var/log/apache2** and **/var/log/mysql** will not exist if logs are using the temporary file system in RAM. Apache will fail to start if these folders are not present. Adding the above lines causes Apache to create these folders when the system boots up.

25. Enable Apache to access files in the the tmpfs **/tmp** folder by backing up and then modifying **/lib/systemd/system/apache2.service** as follows:

```
# changed {date} by admin to enable apache to follow
# symlinks to the /tmp folder in tmpfs
#PrivateTmp=true
PrivateTmp=false
```

This change enables Apache to follow symbolic links in the user document root (public_html) folder to folders and files in the temporary file system.

Reload system deamons

sudo systemctl daemon-reload

Restart Apache service

sudo systemctl restart apache2

This change enables Apache to follow symbolic links in the user document root (public_html) folder to folders and files in the temporary file system **/tmp**.

26. Reboot the Raspberry Pi.

27. Copy the backup file pi_bak.zip from the thumb drive media or other external media to the /home/pi folder. Restore desired files and folders from backup archive by running

Base System Installation Guide

unzip pi_bak.zip

Use **mv** to move folders and files to their appropriate locations.

28. Test all the above modifications.